example

one.Joint control

In this case, the machine will be powered on first, and then the current Angle will be read. After that, the minimum Angle and maximum Angle of joint 1 will be read for single joint Angle control and multi-joint Angle control, and then the specified joint motion, the point motion of absolute position and the point motion of incremental position will be carried out in jog movement. All of the motions mentioned above will print out motion information.

```
int main(int argc, char* argv[]) {
try {
    QCoreApplication a(argc, argv);
    using namespace std::chrono_literals;

    if (!mycobot::MyCobot::I().IsControllerConnected()) {
        std::cerr << "Robot is not connected\n";
        exit(EXIT_FAILURE);
    }

    std::cout << "Robot is connected\n";
    mycobot::MyCobot::I().PowerOn(); // Power on the robot
    mycobot::MyCobot::I().StopRobot(); // Stop the robot
    std::cout << "Robot is moving: " << mycobot::MyCobot::I().IsMoving() << "\n";
// Check if the robot is moving

    mycobot::Angles angles = mycobot::MyCobot::I().GetAngles(); // Get the
current angles of the robot
    std::this_thread::sleep_for(200ms);
    mycobot::Coords coords = mycobot::MyCobot::I().GetCoords(); // Get the
current coordinates of the robot
    angles = mycobot::MyCobot::I().GetAngles(); // Get the current angles of the
robot
    std::cout << "[" << angles[mycobot::J1] << ", " << angles[mycobot::J2] << ",
" << angles[mycobot::J3] << ", "
              << angles[mycobot::J4] << ", " << angles[mycobot::J5] << ", " <<
angles[mycobot::J6] << "]";

    mycobot::MyCobot::I().WriteAngle(mycobot::Joint::J1, 10, 30); // Send single
joint angle

    while (mycobot::MyCobot::I().IsMoving()) { // Check if the robot has reached
the target angle
        angles = mycobot::MyCobot::I().GetAngles();
        std::cout << "Current joint angles" << "[" << angles[mycobot::J1] << ", "
<< angles[mycobot::J2] << ", "
                  << angles[mycobot::J3] << ", " << angles[mycobot::J4] << ", "
                  << angles[mycobot::J5] << ", " << angles[mycobot::J6] << "]" <<
std::flush;
        std::this_thread::sleep_for(200ms);
    }
```

```
    mycobot::Angles goal_angles = { 5, 5, 5, 5, 5, 5 }; // Define a set of angles
    mycobot::MyCobot::I().WriteAngles(goal_angles); // Send joint angles

    while (mycobot::MyCobot::I().IsMoving()) { // Check if the robot has reached
the angles
        angles = mycobot::MyCobot::I().GetAngles();
        std::cout << "Current angles" << "[" << angles[mycobot::J1] << ", " <<
angles[mycobot::J2] << ", "
                  << angles[mycobot::J3] << ", " << angles[mycobot::J4] << ", "
                  << angles[mycobot::J5] << ", " << angles[mycobot::J6] << "]" <<
std::flush;
        std::this_thread::sleep_for(200ms);
    }


    mycobot::MyCobot::I().GetJointMin(mycobot::Joint::J1); // Read the minimum
joint angle
    std::cout << "Current joint angle" << "[" << angles[mycobot::J1]  << "]" <<
std::flush;
    std::this_thread::sleep_for(200ms);

    mycobot::MyCobot::I().GetJointMax(mycobot::Joint::J1); // Read the maximum
joint angle
    std::cout << "Current joint angle" << "[" << angles[mycobot::J1]  << "]" <<
std::flush;
    std::this_thread::sleep_for(200ms);

    // Jog motion on a specified joint
    mycobot::MyCobot::I().JogAngle((mycobot::Joint)0, 100, 5);
    std::this_thread::sleep_for(200ms);
    while (mycobot::MyCobot::I().IsMoving()) { // Check if the motion is
completed
        angles = mycobot::MyCobot::I().GetAngles();
        std::cout << "Performing specified motion, current angle of joint 1" << "
[" << angles[mycobot::J1] << "]"
                  << std::flush;
        std::this_thread::sleep_for(200ms);
    }


    // Jog motion with incremental position on a specified joint
    mycobot::MyCobot::I().JogAngleIncrement((mycobot::Joint)0, 5, 180);
    std::this_thread::sleep_for(200ms);

    while (mycobot::MyCobot::I().IsMoving()) { // Check if the motion is
completed
        angles = mycobot::MyCobot::I().GetAngles();
        std::cout << "Performing incremental motion, current angle of joint 1" <<
"[" << angles[mycobot::J1] << "]"
                  << std::flush;
        std::this_thread::sleep_for(200ms);
    }

    std::this_thread::sleep_for(5000ms); // Delay function
    mycobot::MyCobot::I().StopRobot(); // Stop the robot
```

```
    std::cout << "\n";
    exit(EXIT_SUCCESS); // Exit with success
} catch (std::error_code&) {
    std::cerr << "System error. Exiting.\n";
    exit(EXIT_FAILURE);
} catch (...) {
    std::cerr << "Unknown exception thrown. Exiting.\n";
    exit(EXIT_FAILURE);
}

return 0;
}
```

two.Coordinate control

In this case, the machine will be powered on first, and then the current coordinates will be read. After that, the single coordinate control and multiple coordinate control will be carried out, and then the specified joint coordinate movement in jog motion, the coordinate point movement of absolute position and the coordinate point movement of incremental position will be carried out. All of the motions mentioned above will print out motion information.

```
int main(int argc, char* argv[]) {
try {
    QCoreApplication a(argc, argv);
    using namespace std::chrono_literals;

    if (!mycobot::MyCobot::I().IsControllerConnected()) {
        std::cerr << "Robot is not connected\n";
        exit(EXIT_FAILURE);
    }

    std::cout << "Robot is connected\n";
    mycobot::MyCobot::I().PowerOn(); // Power on the robot
    mycobot::MyCobot::I().StopRobot(); // Stop the robot
    std::cout << "Robot is moving: " << mycobot::MyCobot::I().IsMoving() << "\n";
    mycobot::Angles angles = mycobot::MyCobot::I().GetAngles(); // Get current
joint angles
    std::this_thread::sleep_for(200ms);
    mycobot::Coords coords = mycobot::MyCobot::I().GetCoords(); // Get current
coordinates
    std::cout << "Coordinates before motion" << "[" << coords[mycobot::Axis::X]
<< ", " << coords[mycobot::Axis::Y] << ", " << coords[mycobot::Axis::Z] << ", "
        << coords[mycobot::Axis::RX] << ", " << coords[mycobot::Axis::RY] << ", "
<< coords[mycobot::Axis::RZ] << "]";

    // Send single-parameter coordinates
    mycobot::Coords goal_coords = { 90, 0, 0, 0, 0, 0 }; // Define target
coordinates
    mycobot::MyCobot::I().WriteCoord(mycobot::Axis::X, 90, 180); // Send target
coordinates
    while (!mycobot::MyCobot::I().IsInPosition(goal_coords, false)) {
        coords = mycobot::MyCobot::I().GetCoords(); // Get current coordinates
```

```
        std::cout << "Coordinates after single-parameter motion" << "[" <<
coords[mycobot::Axis::X] << ", " << coords[mycobot::Axis::Y] << ", "
            << coords[mycobot::Axis::Z] << ", " << coords[mycobot::Axis::RX] <<
", "
            << coords[mycobot::Axis::RY] << ", " << coords[mycobot::Axis::RZ] <<
"]" << std::flush;
        std::this_thread::sleep_for(200ms);
    }

    // Send all coordinates
    goal_coords = { 100, 100, 0, 0, 0, 0 }; // Define target coordinates
    mycobot::MyCobot::I().WriteCoords(goal_coords, 30);
    while (!mycobot::MyCobot::I().IsInPosition(goal_coords, false)) {
        coords = mycobot::MyCobot::I().GetCoords(); // Get current coordinates
        std::cout << "Coordinates after all-coordinates motion" << "[" <<
coords[mycobot::Axis::X] << ", " << coords[mycobot::Axis::Y] << ", "
            << coords[mycobot::Axis::Z] << ", " << coords[mycobot::Axis::RX] <<
", "
            << coords[mycobot::Axis::RY] << ", " << coords[mycobot::Axis::RZ] <<
"]" << std::flush;
        std::this_thread::sleep_for(200ms);
    }

    // Jog motion on a specified coordinate axis
    mycobot::MyCobot::I().JogCoord(mycobot::Axis::X, 1, 30);
    while (mycobot::MyCobot::I().IsMoving()) {
        coords = mycobot::MyCobot::I().GetCoords(); // Get current coordinates
        std::cout << "Coordinates after jogging on specified axis" << "[" <<
coords[mycobot::Axis::X] << ", " << coords[mycobot::Axis::Y] << ", "
            << coords[mycobot::Axis::Z] << ", " << coords[mycobot::Axis::RX] <<
", "
            << coords[mycobot::Axis::RY] << ", " << coords[mycobot::Axis::RZ] <<
"]" << std::flush;
        std::this_thread::sleep_for(200ms);
    }

    // Jog motion with incremental position on a specified coordinate axis
    mycobot::MyCobot::I().JogCoordIncrement(mycobot::Axis::X, 30, 30);
    while (mycobot::MyCobot::I().IsMoving()) {
        coords = mycobot::MyCobot::I().GetCoords(); // Get current coordinates
        std::cout << "Coordinates after incremental jogging on specified axis" <<
"[" << coords[mycobot::Axis::X] << ", " << coords[mycobot::Axis::Y] << ", "
            << coords[mycobot::Axis::Z] << ", " << coords[mycobot::Axis::RX] <<
", "
            << coords[mycobot::Axis::RY] << ", " << coords[mycobot::Axis::RZ] <<
"]" << std::flush;
        std::this_thread::sleep_for(200ms);
    }

    mycobot::MyCobot::I().JogAngle(mycobot::Joint::J1, 1, 5);
    std::this_thread::sleep_for(5000ms);
    mycobot::MyCobot::I().StopRobot();

    std::cout << "\n";
    exit(EXIT_SUCCESS);
} catch (std::error_code&) {
```

```
    std::cerr << "System error. Exiting.\n";
    exit(EXIT_FAILURE);
} catch (...) {
    std::cerr << "Unknown exception thrown. Exiting.\n";
    exit(EXIT_FAILURE);
}

return 0;
}
```

tree.IO control

In this example, the machine will be powered on first, the io output will be set, 2, 5, 26 are m5 output pins,35, 36 are m5 input pins, and atom output pins 23, 33 output high level, and read the level information of atom input pins 22, 19.

```
int main(int argc, char* argv[])
{
    try {
        QCoreApplication a(argc, argv);
        using namespace std::chrono_literals;
        if (!mycobot::MyCobot::I().IsControllerConnected()) {
            std::cerr << "Robot is not connected\n";
            exit(EXIT_FAILURE);
    }
    std::cout << "Robot is connected\n";
    mycobot::MyCobot::I().PowerOn();
    mycobot::MyCobot::I().SleepSecond(1);

    mycobot::MyCobot::I().SetBasicOut(2, 1);
    mycobot::MyCobot::I().SleepSecond(1);
    mycobot::MyCobot::I().SetBasicOut(5, 1);
    mycobot::MyCobot::I().SleepSecond(1);
    mycobot::MyCobot::I().SetBasicOut(26, 1);
    mycobot::MyCobot::I().SleepSecond(1);

    for (int i = 0; i < 2; i++) {
        std::cout << "35= " << mycobot::MyCobot::I().GetBasicIn(35) << std::endl;
        mycobot::MyCobot::I().SleepSecond(1);
        std::cout << "36= " << mycobot::MyCobot::I().GetBasicIn(36) << std::endl;
        mycobot::MyCobot::I().SleepSecond(1);
    }

    /*mycobot::MyCobot::I().SetDigitalOut(23, 1);
    mycobot::MyCobot::I().SleepSecond(1);
    mycobot::MyCobot::I().SetDigitalOut(33, 1);
    mycobot::MyCobot::I().SleepSecond(1);*/

    for (int i = 0; i < 2; i++) {
        std::cout << "22= " << mycobot::MyCobot::I().GetDigitalIn(22) <<
std::endl;
        mycobot::MyCobot::I().SleepSecond(1);
        std::cout << "19= " << mycobot::MyCobot::I().GetDigitalIn(19) <<
std::endl;
        mycobot::MyCobot::I().SleepSecond(1);
```

```
    }

    mycobot::MyCobot::I().StopRobot();
    std::cout << "Robot is moving: " << mycobot::MyCobot::I
    std::this_thread::sleep_for(5000ms);
    mycobot::MyCobot::I().StopRobot();

    std::cout << "\n";
    exit(EXIT_SUCCESS);
    } catch (std::error_code&) {
    std::cerr << "System error. Exiting.\n";
    exit(EXIT_FAILURE);
    } catch (...) {
    std::cerr << "Unknown exception thrown. Exiting.\n";
    exit(EXIT_FAILURE);
    }
}
```